

Sorbonne Université
MU4IN019 – Nuage et Réseaux Virtuels

TME 2 – 3

Support de la conteneurisation sous Linux
et divers utilitaires

Auteurs :

HUANG Loïc (21203938)
HARRAOUI Ines (21204796)

Année universitaire 2025–2026

Table des matières

1	I. Inspection du code source : conception du noyau pour la conteneurisation	2
2	II. Création et manipulation d'espaces de noms avec les utilitaires Sys-Utils	4
3	III. Espaces de noms réseaux et l'utilitaire ip netns	8
4	IV. Contrôle des ressources : cgroups	11
5	V. Docker et les espaces de noms	12

I. Inspection du code source : conception du noyau pour la conteneurisation

1. Fichier nsproxy.h

- a. *Comparez la liste des espaces de noms (namespaces) mentionnés dans `nsproxy.h` avec celui listé dans le manuel (`man namespaces`).*

Réponse

La liste est identique à celle de `man namespaces`. Cependant, le namespace **User** est absent du fichier `nsproxy.h`.

- b. *Pourquoi la structure `pid_namespace` s'appelle-t-elle `pid_ns_for_children` ?*

Réponse

Parce qu'elle permet d'indiquer aux **processus fils** du processus courant dans quel namespace ils devront se trouver lors de leur création.

2. Site <https://www.kernel.org/> – Fichier `fork.c` : implémentation de `unshare()`

- a. *Quels sont les arguments d'entrée de l'appel système `unshare()` ? Si l'on veut annuler le partage de la pile réseau, quelle doit être la valeur de l'argument ?*

Réponse

L'argument est une valeur `unsigned long` nommée `unshare_flags`. Pour annuler le partage de la pile réseau, il faut utiliser la valeur `CLONE_NEWNET`.

Image 1 : Fonction d'appel système `unshare()`

- b. *Trouvez l'appel de fonction qui déclenche la création d'une pile réseau distincte. Quelle est la variable qui pointera vers le nouvel espace de noms réseau ?*

Réponse

Fonction : `unshare_nsproxy_namespaces`

Variable : `new_ns_proxy`

- c. *Trouvez les deux lignes qui changent l'espace de noms réseau vers le nouvel espace de noms créé.*

Réponse

Ces deux lignes effectuent le changement effectif de l'espace de noms réseau vers le nouvel espace créé.

Image 2 : Appel de la fonction `switch_task_namespaces`

3. Fichier `net_namespace.h`

- a. Trouvez la ligne qui indique la liste des espaces de noms réseau maintenus par le noyau.

Réponse

La **ligne 63** indique la liste des espaces de noms réseau maintenus par le noyau.

Image 3 : Variable list

- b. Les structures de données IPv4 sont-elles différentes d'un espace de noms à un autre ? Justifiez.

Réponse

Oui : à chaque création de namespace, une **nouvelle instance de la variable ipv4** est initialisée, indépendante des autres namespaces.

- c. Quelle est la fonction appelée pour ajouter des opérations d'initialisation et de terminaison d'un module d'espace de noms réseau ?

Réponse

La fonction `register_pernet_subsys()`.

4. Fichier `icmp.c`

- a. La fonction identifiée en 3c est-elle appelée dans ce fichier ?

Réponse

Oui, `register_pernet_subsys()` est bien présente et utilisée dans plusieurs fonctions et structures du fichier.

- b. Quel est le nom de la fonction d'initialisation du module ICMP ? Quelle est la fonction de sortie ?

Réponse

- Initialisation : `icmpv6_init(void)`
- Sortie : `icmpv6_cleanup(void)`

- c. Pourquoi la plupart des variables du fichier `icmp.h` sont-elles précédées par `net->` ?

Réponse

Car il faut **préciser dans quel namespace** le module ICMP doit opérer. Sans `net->`, il serait impossible de savoir à quel namespace le module est rattaché.

5. Fichier `sys-utils/unshare.c`

- a. Trouvez la ligne où l'appel système `unshare()` est utilisé pour modifier les espaces de noms.

Réponse

Ligne **1112** du fichier.

Image 4 : Utilisation de la fonction unshare()

- b. Quel est le drapeau `unshare_flag` qui crée un espace de noms réseau ? À quelle option correspond-il ? Même question pour l'espace de noms PID et l'espace de noms de montage.

Réponse

Espace de noms	Drapeau	Option
Réseau	CLONE_NEWNET	-n
PID	CLONE_NEWPID	-p
Montage	CLONE_NEWNS	-m

Image 5 : Liste des options de la commande unshare

II. Création et manipulation d'espaces de noms avec les utilitaires Sys-Utls

1. Utilisez le PID du processus `bash` pour exécuter `ls -al /proc/PID/ns`. La liste des espaces de noms correspond-elle à `nsproxy.h` et `man namespaces` ?

Réponse

```
ps                                # Etape 1
# noter le PID du bash
ls -al /proc/<PID_BASH>/ns        # Etape 3
```

- a. La liste est identique à `nsproxy.h`. Le namespace **User** est toujours absent par rapport à `man namespaces`.

- b. Faites de même avec `PID=1` : `ls -al /proc/1/ns`. Pourquoi les numéros d'inodes sont-ils identiques ?

Réponse

Les inodes sont identiques car le PID courant et le PID 1 **partagent le même namespace** (namespace initial du système).

Image 6 : Utilisation des commandes pour les questions 1

2. Exécutez consécutivement : `ps aux`, `unshare -pid -fork -mount-proc bash -l, xclock &`, `ps aux`. Décrivez les résultats.

Réponse

Avant : `ps aux` affiche tous les processus du système.

Après : `ps aux` n'affiche que les processus du **nouveau namespace PID**, grâce au `-mount-proc` qui monte un `/proc` isolé.

Image 7 : `ps aux` avant l'exécution des commandes

Image 8 : Exécution de la commande puis vérification du `ps aux`

a.

Quel processus est le processus d'initialisation du nouvel espace de noms PID ? Quel est le nouveau PID de `xclock` ?

b.

Réponse

Processus d'initialisation (PID 1 dans le nouveau namespace) : `bash -l`

Nouveau PID de `xclock` dans le namespace : **17**

Depuis un autre terminal, tapez `ps aux | grep "bash -l"` et `ps aux | grep xclock`. Répondez aux sous-questions.

c.

Réponse

```
ps aux | grep "bash -l"
ps aux | grep xclock
```

Image 9 : Exécution des commandes ci-dessus

1. PID de `bash -l` (espace initial) : **3719**
2. PID du processus exécutant `unshare()` : **3718**
3. PID de `xclock` (espace initial) : **3735**
4. Les inodes du namespace PID sont **différents** entre `unshare()` (PID 3718) et `bash -l/xclock` : `unshare()` s'exécute dans le namespace initial, tandis que `bash -l` et `xclock` s'exécutent dans le **nouveau namespace créé**.

Image 10 : Inodes de `bash -l`

Image 11 : Inodes de `unshare()`

Image 12 : Inodes de `xclock`

Depuis un troisième terminal, utilisez `nsenter -p -m -t PID_cible` puis `ps aux` et `ls /proc`.

d.

Réponse

```
nsenter -p -m -t <PID_cible>
ps aux
ls /proc
```

Les PIDs affichés doivent être identiques à ceux vus depuis le terminal `unshare`.

- e. *Pourquoi l'option `-m` de `nsenter` est-elle requise ?*

Réponse

L'option `-m` permet d'afficher le `/proc` du **conteneur cible**. Sans elle, on verrait tous les processus du `/proc` de la machine hôte.

- f. *Testez la commande `lsns` et ses variantes.*

Réponse

```
lsns
lsns -t net
lsns -t pid
lsns -p <PID_cible>
lsns -o PATH,PID,PPID
```

`lsns` affiche la liste des espaces de noms et leurs informations (PID, PPID, chemin, type...) de façon lisible.

Image 13 : Exécution des commandes `lsns`

Image 14 : Exécution de la commande `lsns -o Path,PID,PPID`

4. *Pour chaque commande du bloc suivant, fournissez une description.*

Réponse

```
mkdir /mesconteneurs
mkdir /mesconteneurs/pc1
mount --bind --make-private /mesconteneurs/pc1 /mesconteneurs/pc1
touch /mesconteneurs/pc1/mnt /mesconteneurs/pc1/net \
      /mesconteneurs/pc1/pid /mesconteneurs/pc1/uts
unshare --fork --mount-proc \
  --mount=/mesconteneurs/pc1/mnt \
  --net=/mesconteneurs/pc1/net \
  --uts=/mesconteneurs/pc1/uts \
  --pid sleep infinity 1 &
cpid=$(pgrep -fn "sleep infinity 1")
mount --bind /proc/$cpid/ns/pid /mesconteneurs/pc1/pid
```

- `mkdir /mesconteneurs` : crée le répertoire à la **racine** du système (grâce au `/`). Sans `/`, le répertoire serait créé dans le répertoire courant.
- `mkdir /mesconteneurs/pc1` : crée le sous-répertoire `pc1`.
- `mount -bind -make-private ...` : crée un point de montage sur lui-même. `-bind` permet d'ajouter `-make-private` qui rend tous les montages intérieurs **invisibles** depuis l'extérieur du namespace.
- `touch ... mnt net pid uts` : crée quatre fichiers vides servant de points d'ancrage pour les bind mounts des namespaces.
- `unshare -fork ...` : crée un processus fils avec des copies des namespaces `mount`, `net` et `uts`. Le `sleep infinity 1` maintient le processus actif pour que les namespaces persistent. Sans lui, le processus fils se terminerait et les namespaces disparaîtraient.
- `cpid=$(pgrep -fn "sleep infinity 1")` : récupère le PID du processus `sleep infinity 1`.
- `mount -bind /proc/$cpid/ns/pid ...` : bind mount entre le namespace PID du processus fils et le fichier `pid`, rendant le namespace **persistant et nommé**.

Pour entrer dans le conteneur :

```
nsenter --net=/mesconteneurs/pc1/net \
  --mount=/mesconteneurs/pc1/mnt \
  --pid=/mesconteneurs/pc1/pid \
  --uts=/mesconteneurs/pc1/uts bash
```

- a. Un montage depuis le namespace `pc1` rend le contenu de `/mesconteneurs/pc1/d1` visible dans `/root/d1` depuis l'intérieur de `pc1`, mais ce montage reste **invisible depuis l'extérieur** du namespace.
- b. Pour vérifier la connectivité et utiliser Wireshark, les interfaces doivent avoir une adresse IP et être activées (contrairement à FreeBSD, ce n'est pas automatique sous Linux) :

```
ip link set dev <interface> up
```

- c. Pour déplacer `seth1` dans le namespace hôte :

III. Espaces de noms réseaux et l'utilitaire `ip netns`

1. Créez deux espaces de noms réseaux nommés `ns3` et `ns4`.

Réponse

```
ip netns add ns3
ip netns add ns4
```

2. Quelle est la liste des espaces de noms (`ip netns list`) ? Que représentent les autres lignes en dehors de `ns3` et `ns4` ?

Réponse

La liste contient `ns3`, `ns4`, `pc1` et `pc2`. Les autres lignes représentent les namespaces persistants créés à la section II.

3. Lancez un `bash` dans `ns3` et `ns4`. Modifiez `/etc/resolv.conf` depuis `ns3` : la modification apparaît-elle dans `ns4` ? Rendez les deux fichiers différents.

Réponse

```
ip netns exec ns3 bash
```

Oui, la modification de `/etc/resolv.conf` depuis `ns3` apparaît dans `ns4` car ils partagent le même fichier par défaut. Pour les rendre différents :

```
mkdir -p /etc/netns/ns3
mkdir -p /etc/netns/ns4
echo "je suis dans ns3" > /etc/netns/ns3/resolv.conf
echo "je suis dans ns4" > /etc/netns/ns4/resolv.conf
```

Les modifications ne se propagent plus entre les namespaces.

4. Lancez `xclock`, `xterm`, `xeyes`, `xedit` dans `ns3/ns4`. Vérifiez avec `ip netns pids`.

Réponse

```
ip netns exec ns3 xclock &
ip netns exec ns3 xterm &
ip netns exec ns4 xeyes &
ip netns exec ns4 xedit &
ip netns pids ns3
ip netns pids ns4
```

Les PIDs affichés correspondent bien aux processus lancés dans ns3 et ns4.

Image : liste des PIDs sur ns3 et ns4

Image : PIDs des processus sur les 2 namespaces

Utilisez le dernier PID affiché par `ip netns pids ns3` avec `ip netns identify`. Commentez les résultats.

a.

Réponse

```
ip netns identify 6868      # -> "ns3"
ip netns identify 1        # -> rien (namespace initial)
ip netns identify 1112229  # -> erreur (PID inexistant)
```

Testez `ip netns monitor` depuis un terminal puis ajoutez/supprimez des namespaces depuis un autre. Déduisez son rôle.

b.

Réponse

`ip netns monitor` surveille **en temps réel** la création et suppression de namespaces : affiche `add <nom>` ou `del <nom>` à chaque événement.

Testez `ip netns set ns3 33`, `ip netns set ns4 44`, `ip netns list-id`. À quoi servent les options `list-id` et `set` ?

c.

Réponse

```
ip netns set ns3 33
ip netns set ns4 44
ip netns list-id
```

`list-id` affiche les namespaces avec leur identifiant numérique (`netnsid`) ; `set` permet d'en attribuer un manuellement. Ces identifiants facilitent l'automatisation.

Créez deux liens `veth eth3-seth3` et `eth4-seth4`. Associez `eth3` à `ns3` et `eth4` à `ns4`. Configurez et testez.

5.

Réponse

```
ip link add eth3 type veth peer name seth3
ip link add eth4 type veth peer name seth4
ip link set eth3 netns ns3
ip link set eth4 netns ns4
ip netns exec ns3 ip addr add 128.0.0.3/24 dev eth3
ip netns exec ns4 ip addr add 128.0.0.4/24 dev eth4
```

Connectivité vérifiée par ping entre ns3 et ns4.

6. Créez un bridge *monbr*. Rattachez *seth1*, *seth2*, *seth3*, *seth4* au bridge. Testez la connectivité entre *pc1*, *pc2*, *ns3* et *ns4*.

Réponse

```
ip link add monbr type bridge
ip link set monbr up
ip link set seth1 up && ip link set seth1 master monbr
ip link set seth2 up && ip link set seth2 master monbr
ip link set seth3 up && ip link set seth3 master monbr
ip link set seth4 up && ip link set seth4 master monbr

ip netns exec pc1 ping 128.0.0.2    # -> Marche
ip netns exec pc1 ping 128.0.0.3    # -> Marche
ip netns exec pc1 ping 128.0.0.4    # -> Marche
```

7. Détachez *eth3* et *eth4*, puis créez deux interfaces *macvlan* en mode Bridge (*macvlan3*, *macvlan4*) rattachées à *enp0s3*. Déplacez-les vers *ns3* et *ns4*. Testez.

Réponse

```
# Recuperer les interfaces depuis les namespaces
ip netns exec ns3 ip link set eth3 netns 1
ip netns exec ns4 ip link set eth4 netns 1

# Creer les interfaces macvlan
ip link add macvlan3 link enp0s3 type macvlan mode bridge
ip link add macvlan4 link enp0s3 type macvlan mode bridge
ip link set macvlan3 netns ns3
ip link set macvlan4 netns ns4

# Configuration ns3
ip netns exec ns3 ip addr add 128.0.0.103/24 dev macvlan3
ip netns exec ns3 ip link set macvlan3 up
ip netns exec ns3 ip link set lo up

# Configuration ns4
ip netns exec ns4 ip addr add 128.0.0.104/24 dev macvlan4
ip netns exec ns4 ip link set macvlan4 up
ip netns exec ns4 ip link set lo up

# Tests
ip netns exec ns3 ping -c 2 128.0.0.104      # -> Marche
ip netns exec ns4 ping -c 2 128.0.0.103      # -> Marche
```

IV. Contrôle des ressources : cgroups

Installez *stress*. Lancez *stress -v -vm-bytes 13M -vm-keep -vm 1* et observez */proc/stressPID/status*.

1.

Réponse

```
apt install stress
stress -v --vm-bytes 13M --vm-keep --vm 1
cat /proc/<stressPID>/status
```

Le processus s'exécute et alloue 13 Mo de RAM sans contrainte à ce stade.

2.

Créez un cgroup *pc1* dans le contrôleur mémoire et limitez-le à 10 Mo.

Réponse

```
mkdir /sys/fs/cgroup/memory/pc1
ls /sys/fs/cgroup/memory/pc1
cd /sys/fs/cgroup/memory/pc1
echo $((10 * 1024 * 1024)) > memory.limit_in_bytes
```

La limite mémoire du groupe est fixée à **10 Mo**.

3. Ajoutez le `bash` au `cgroup`, relancez `stress` et observez `cgroup.procs`, `/proc/stressPID/status` et `ps aux`.

Réponse

```
echo $$ # PID du bash cible
echo <PID> > cgroup.procs
stress -v --vm-bytes 13M --vm-keep --vm 1
```

Le processus est limité à **10 Mo de RAM**. Le surplus (3 Mo) est redirigé vers le **swap**. Sans swap activé, le processus ne pourrait pas s'exécuter. Le RSS dans `ps aux` reste toujours inférieur à 10.

V. Docker et les espaces de noms

1. Lancez le conteneur `dctest` avec `docker run -d -p 8088:80 -name dctest docker/getting-started`. Répondez aux sous-questions.

- a. Expliquez et comparez les résultats de `ls -al /proc/premierPID/ns`, `ls -al /proc/deuxiemePID/ns`, `ls -al /proc/1/ns`, `ps aux | grep nginx`, `docker exec dctest ps aux`.

Réponse

Les deux processus Docker partagent les mêmes inodes de namespace *entre eux*, mais diffèrent du PID 1 pour `mnt`, `net`, `pid`, `ipc` (en partageant `cgroup` et `user`). Les processus Docker sont visibles depuis l'hôte via `ps aux`, mais depuis le conteneur, le PID 1 correspond au processus `master` (PID **8714** depuis l'hôte).

- b. Expliquez les résultats de `docker exec dctest hostname`, `nsenter -u -t premierPID hostname dctest`, puis `docker exec dctest hostname`.

Réponse

Le `hostname` initial du conteneur est `3f32d31a9636`. On le modifie en `dctest` via `nsenter` (Docker n'a pas la permission de modifier le `hostname` directement) :

```
nsenter -u -t <premierPID> hostname dctest
docker exec dctest hostname # -> dctest
```

Cela illustre que tout ce qui se passe dans un conteneur peut être **observé ou modifié** par un utilisateur `root` sur l'hôte via l'accès aux namespaces.

- c. Exécutez `ps -eo pid,ppid,command f` et répondez aux sous-questions sur `runc`.

1. Quel est le processus père du serveur web ? Quel est son propre père ?

Réponse

- **1^{ère} ligne** = père du père du serveur web (runtime runc)
- **2^{ème} ligne** = père direct du serveur web

Vérifiez le runtime (`docker info | grep -i runtime`). Consultez le fichier `config.json` du conteneur.

- 2.
- a) Quels sont les espaces de noms créés par `runc` ?
 - b) Quel est l'emplacement réel du fichier `/etc/resolv.conf` du conteneur ?

Réponse

- a. Namespaces créés par `runc` :
 - `mount`, `network`, `uts`, `pid`, `ipc`
- b. Emplacement réel du `resolv.conf` :
`/var/lib/docker/containers/3f...92c/resolv.conf`

Trouvez le répertoire runtime avec `ps aux | grep runtime-root`.

- 3.
- a) Quel répertoire `cgroupfs` contrôle les ressources du conteneur ?
 - b) Quels CPUs peut utiliser le conteneur ?
 - c) Que représente la liste de PIDs dans `cgroup.procs` ?

Réponse

- a. Chemin `cgroupfs` :
`/sys/fs/cgroup/cpuset/docker/3f32d31a...a7685392c/`
- b. Tous les processeurs sont disponibles (processeur 0, 1, etc.).
- c. La liste de PIDs dans `cgroup.procs` représente les **processus Docker vus depuis l'hôte**.

2. Réseaux de conteneurs Docker

Créez un bridge Docker `mydefaultbridge`, lancez `dc5` et `dc6`, connectez-les. Docker utilise-t-il la même technique que les slides (`veth` + `bridge`) ? Prouvez-le.

a.

Réponse

```
docker network create -d bridge mydefaultbridge
docker run -d --net mydefaultbridge --name dc5 \
    docker/getting-started
docker run -d --name dc6 docker/getting-started
docker network connect mydefaultbridge dc6
docker exec dc5 ping <adresse_dc6>

# Preuve : les interfaces veth sont connectees au bridge
ip a | grep ": br-"
ip a | grep "master br"
```

Les deux interfaces veth sont bien connectées au bridge Docker, confirmant l'utilisation de la même technique (veth + bridge) que celle décrite dans le cours 2.

- b. *Utilisez **nomaster** pour détacher les interfaces veth de mydefaultbridge.*

Réponse

```
# Renommer les veth en noms simples (optionnel)
ip link set <veth_dc5> name seth5
ip link set <veth_dc6> name seth6
# Detacher du bridge
ip link set seth5 nomaster
ip link set seth6 nomaster
```